Kryptonite-3.0: The Final End to Machine Learning Hype?

Harley Quinn [1] **Lex Luther** [2]

Abstract

Last year, our challenge dataset Kryptonite-2.0 showed that a simple binary classification task could defeat polynomial basis expansion and GPT embeddings alike. The response from the community was predictable: our datasets were too small, our models were too weak, and modern foundation models would surely succeed where logistic regression had failed. In this work we answer every one of these objections. We introduce Kryptonite-3.0, a redesigned, procedurally generated dataset with an n dimensional feature space and binary label space, released for $n \in \{20, 30, 40\}$ with up to 32,000 labelled examples each. We find that polynomial basis expansion never exceeds 59% accuracy, even at degree 10 with more than ten billion features, that quadrupling the amount of training data does not change this conclusion, and that KryptoNet-40, a large foundation model pretrained on 200,000 examples of a closely related task, scores worse than a coin flip, a result its learned representation cannot improve on. The universal function approximation argument, the more-data argument and the foundation model argument therefore all fall to a problem that fits on a single page.

1. Introduction

In recent years, machine learning (ML) has become the dominant paradigm for solving complex, data-driven problems across a wide array of industries. From financial services (Heaton et al., 2017) to medical diagnosis (Esteva et al., 2019) and autonomous navigation (Bojarski et al., 2016), the versatility and success of machine learning models are undeniable. Driven by advances in hardware, data availability, and algorithmic breakthroughs, ML models have grown increasingly complex, with larger architectures tackling ever more challenging tasks (Bommasani et al., 2021).

Amidst these developments, the hype surrounding machine learning has grown exponentially. Some researchers and technologists claim that we are approaching a point where models may achieve artificial general intelligence (AGI) (Bubeck et al., 2023), a form of intelligence that can perform any intellectual task that a human can. Supporting these claims are theoretical results, such as the Universal Approximation Theorem (Hornik, 1991), which suggests that neural networks can approximate any continuous function; empirical scaling laws (Kaplan et al., 2020), which suggest that performance improves predictably with more data; and the rise of foundation models (Bommasani et al., 2021), large pretrained models that are claimed to adapt to almost any downstream task with only a little fine-tuning.

However, the remarkable growth and capabilities of ML models should be approached with caution. Recently, pertinent stream of research seeks to simultaneously challenge and spur important growth in machine learning models: challenge datasets (Chollet, 2019; Küttler et al., 2020; Sawada et al., 2023). Such datasets are carefully crafted such that modern machine learning algorithms struggle to achieve strong performance. Our own Kryptonite-2.0 (Quinn & Luther, 2025) showed that even a simple binary classification problem can be one of them. Critics of that work argued that its datasets were too small, its models were too weak, and that a model pretrained on data like ours would succeed where a general-purpose language model had failed.

In this work, we introduce Kryptonite-3.0, a redesigned and harder version of our dataset that addresses each of these criticisms directly. Comprising an n-dimensional feature space mapped onto a binary label space, Kryptonite-3.0 remains the simplest challenge dataset that addresses both theoretical and empirical claims underpinning machine learning hype, and it now ships with its very own foundation model, KryptoNet-40.

We begin by showing that Kryptonite-3.0 is a counterexample to the universal function approximation theorem: polynomial basis expansion up to degree 10, with more than ten billion features, fails to classify the data for every value of n we consider. We then show that the problem is not one of data, as quadrupling the size of the training set

[1]Wayne University of Metropolis, Gotham, United States [2]Xavier Academy for Advanced Sciences, Westchester, United States. Correspondence to: Harley Quinn, PhD <No email given>.

Coursework. For review by instructors of Mathematics for Machine Learning (70015) . Do not distribute.

Math for ML (70015) Coursework Prompt

leaves accuracy essentially unchanged. Finally, we show that a foundation model pretrained on 200,000 examples of a closely related task is worse than random guessing on Kryptonite-3.0, and that neither a linear classifier nor a small neural network trained on its learned representation does any better. These results cast doubt on the sweeping claims of universality, scalability and general intelligence in machine learning models and highlight critical limitations that must be addressed if we are to move forward in a meaningful way.

We begin by discussing the most pertinent related works to Kryptonite-3.0, then we cover our experimental design and results for invalidating the universal function approximation theorem and the benefit of more data. We then describe our experiments with KryptoNet-40. We open source all of our experiments, the datasets, the pretrained model, and an unlabeled set of data to ultimately benchmark potential challenge solutions. We conclude the paper with prospective approaches for solving the Kryptonite-3.0 tasks. As with Kryptonite-2.0, we feel that it is ultimately a useless exercise for future researchers and that we should all abandon machine learning research in favor of... something else? Perhaps Kryptonite-4.0.

2. Related Works

~~It is widely accepted that the creation and curation of challenging datasets is a valuable modality of contribution to AI research. One prominent example of this is the Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) benchmark and corresponding prize (Chollet, 2019). This benchmark procedurally generates complex logic puzzles which are currently outside of the scope for foundation models. This approach has been extended to groups of concepts (Moskvichev et al., 2023) as well as directly to text where the authors attempt to make them natural language centric (Sawada et al., 2023). Another example of a prominent challenge dataset is the NetHack challenge (Küttler et al., 2020) which requires models to reason about a text-based adventure game.~~

The closest work to ours is Kryptonite-2.0 (Quinn & Luther, 2025), which introduced the Kryptonite-n family and evaluated polynomial basis expansion and frozen GPT-2 embeddings. This work differs in three ways: the datasets are larger and higher dimensional, we study the effect of training set size, and we evaluate a foundation model pretrained on data like our own rather than a general-purpose language model. Unlike the abstract reasoning benchmarks above, Kryptonite-3.0 remains a very simple, binary classification problem that also proves remarkably challenging for machine learning models.

3. Methodology

In order to preserve the challenge of the dataset we would like to avoid models that are *a priori* suited to the task i.e., models whose features space is tailored to the task at hand. Thus, as with Kryptonite-2.0, we will not release the exact inner working of Kryptonite-3.0. We do specify that there *is* an underlying pattern to the feature-label pair. That is to say the dataset is not *impossible*, just designed to be difficult for machine learning models. We denote a machine learning model as a parametric function f with parameters $\theta \in \Theta$, which maps from features $x \in \mathbb{R}^n$ to labels $y \in \{0, 1\}$. We consider supervised learning in the classification setting with a labeled dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$. The model parameters are trained, starting from some initialization θ' denoted as M, as $\theta = M(f, \theta', \mathcal{D})$. In other words, given a model, initialization, and dataset, the training function M returns the "trained" parameters θ. The function M is typically taken to be stochastic with randomness stemming from the initialization, batch ordering, and any noise added.

3.1. Binary Logistic Regression and Gradient-Based Training

Binary logistic regression is one of the most well-investigated machine learning models for binary classification, aiming to estimate the probability that a given input $\mathbf{x} \in \mathbb{R}^n$ belongs to one of two classes, typically denoted $y = 0$ or $y = 1$. Logistic regression models the probability of class membership as a function of the input features:

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \sigma(\mathbf{x}^\top \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\mathbf{x}^\top \boldsymbol{\theta})},$$

where $\boldsymbol{\theta} \in \mathbb{R}^n$ is the parameter vector and $\sigma(\cdot)$ denotes the logistic sigmoid function. The objective typically used to learn this model is the categorical cross-entropy loss which we will denote generically with $\mathcal{L}$.

Formulating the problem of learning as an optimization problem for a dataset $\mathbf{X} \in \mathbb{R}^{N \times n}$ with corresponding labels $\mathbf{y} \in \{0, 1\}^N$, binary logistic regression does not admit a closed form solution. Instead, we use gradient-based methods to learn the model. This is particularly appropriate in our setting as gradient descent (and its variants) is the algorithm used to learn in some of the most prominent examples of machine learning successes. Gradient descent can be stated as an iterative process with the equations:

$$\boldsymbol{\theta}^{(1)} \sim \mathcal{N}(0, \mathbf{I}\sigma^2) \quad \text{random initialization of theta}$$
$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \alpha \left(\nabla_{\boldsymbol{\theta}} \mathcal{L} \left(\boldsymbol{\theta}^{(i)} \right) \right)^\top \tag{1}$$

Where α is called the learning rate and we run the algorithm for a fixed number of epochs.

The gradient of the objective function in the case of our loss function respect to $\boldsymbol{\theta}$ in its vectorized form, which in

Math for ML (70015) Coursework Prompt

numerator layout is a row vector, is:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = (\sigma(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y})^\top \mathbf{X} \in \mathbb{R}^{1\times n},$$

where $\sigma(\mathbf{X}\boldsymbol{\theta})$ denotes the element-wise application of the sigmoid function across the predictions for each sample. By iteratively updating $\boldsymbol{\theta}$ in the direction of the negative gradient using methods such as stochastic gradient descent (SGD) or Adam, the model converges to parameter values that improve prediction accuracy over the training data.

3.2. Basis Expansion and Universal Function Approximation

While the linear form of logistic regression is effective for linearly separable data, it may struggle with more complex decision boundaries. Basis expansion is a powerful extension that enhances the model's expressive capacity by applying a nonlinear transformation to the input features. This is done by mapping each input $\mathbf{x}$ to a higher-dimensional feature space $\phi(\mathbf{x})$, where $\phi(\cdot)$ represents a set of basis functions such as polynomial, radial basis, or kernel functions. Logistic regression with basis expansion thus models the probability of the positive class as:

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \sigma(\phi(\mathbf{x})^\top \boldsymbol{\theta}).$$

By choosing a sufficiently rich set of basis functions, the model can approximate a wide range of complex decision boundaries. In fact, basis expansion enables logistic regression to achieve universal function approximation, meaning that with an adequate choice of basis functions, the model can approximate any continuous function on a compact domain arbitrarily well (De Branges, 1959). For the polynomial basis of degree d on n inputs, the number of features is

$$m = \binom{n+d}{d},$$

which grows rapidly with d: for $n = 40$ and $d = 10$ it exceeds ten billion. Fortunately, the kernel trick lets us fit the same model without ever forming $\phi(\mathbf{x})$. Writing $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$, the trained model takes the form

$$P(y = 1|\mathbf{x}) = \sigma\Big(\sum_{i=1}^{N} a_i\, k(\mathbf{x}^{(i)}, \mathbf{x})\Big),$$

so only inner products between examples are needed. This allows us to test polynomial basis expansions of far higher degree than previous work. Such powerful theoretical statements are sometimes used as the basis for making claims that machine learning models have limitless potential. In our experiments we will test this sort of reasoning.

3.3. Pre-Trained Foundation Models

The modern answer to any difficult learning problem is to start from a large model that has already been pretrained on related data and then adapt it to the task at hand (Bommasani et al., 2021). To give the foundation model paradigm the fairest possible chance on our dataset, we pretrain our own foundation model, KryptoNet-40, a deep multilayer perceptron with four hidden layers of width 512:

$$\begin{aligned} \mathbf{h}^{(0)} &= \mathbf{x}, \\ \mathbf{h}^{(l)} &= \mathrm{ReLU}\left(\mathbf{W}_l \mathbf{h}^{(l-1)} + \mathbf{b}_l\right), \quad l = 1, \ldots, 4, \\ f_{\boldsymbol{\theta}}(\mathbf{x}) &= \sigma\left(\mathbf{w}_5^\top \mathbf{h}^{(4)} + b_5\right). \end{aligned} \tag{2}$$

In total, KryptoNet-40 has 809,473 parameters. It is pretrained with Adam on 200,000 labelled examples of Kryptonite-40-Pre, a closely related task produced by the same generator as Kryptonite-40, where it reaches 95% accuracy.

We consider three standard ways of using such a model on a new task. In the *zero-shot* setting the pretrained parameters $\boldsymbol{\theta}_{\text{pre}}$ are used as they are. In the *linear probe* setting, the hidden layers are frozen and only the final layer $(\mathbf{w}_5, b_5)$ is retrained with Equation (1), so that the pretrained network acts as a learned basis expansion $\phi(\mathbf{x}) = \mathbf{h}^{(4)}$. In the *MLP probe* setting, the hidden layers are again frozen and the final layer is replaced by a small neural network with one hidden layer of 512 units, which can recover information that is present in $\mathbf{h}^{(4)}$ but not linearly accessible. We do not update the hidden layers: with only a few thousand labelled examples, retraining a model with 809,473 parameters would inevitably overfit.

4. Experiments

In this section, we experimentally evaluate the performance of logistic regression when varying the number of basis features and the amount of training data, and we evaluate our pretrained foundation model KryptoNet-40 under zero-shot use, linear probing and MLP probing. All of the code for our experiments can be found on github.com.

4.1. Dataset Descriptions

As previously noted each version of Kryptonite-3.0 has an n dimensional feature space and represents a binary classification problem. As with Kryptonite-2.0, the number of samples increases with the dimensionality of each dataset, because keeping the number of examples fixed would make the problem impossible as n goes to infinity. We additionally release Kryptonite-40-Adapt, a small dataset for adapting KryptoNet-40. For every dataset we also release 10,000 unlabelled examples on which potential solutions can be benchmarked. The datasets are summarised in Table 1.

Math for ML (70015) Coursework Prompt

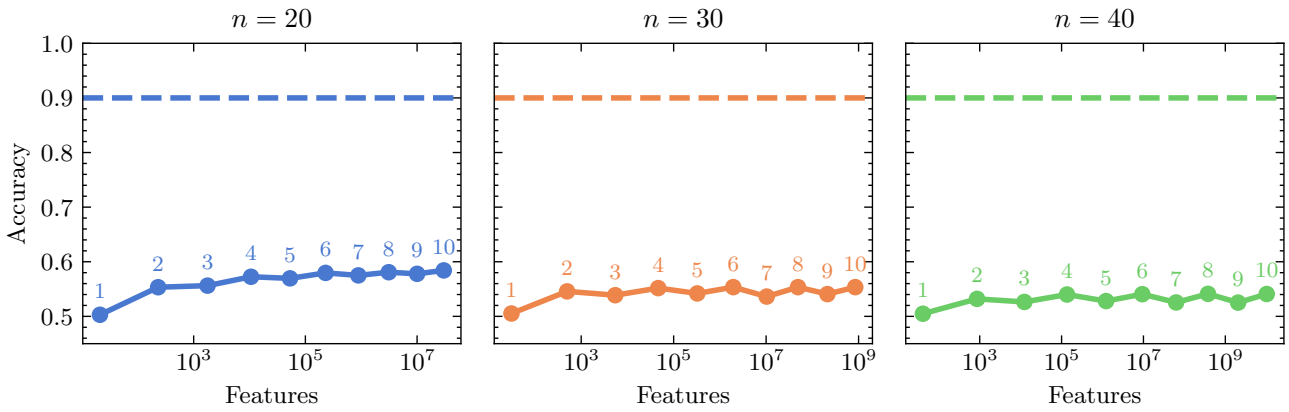


Figure 1. Classification accuracies on the Kryptonite-n datasets using polynomially-expanded logistic regression across various degrees of polynomial expansion (marked above each point). Notably, no configuration of n or polynomial degree achieves an accuracy above 59%, illustrating the model's limitations in capturing the dataset's complex structures despite increasing the number of features to over ten billion. Target accuracies are shown as dashed lines.

Table 1. The Kryptonite-3.0 datasets and their target accuracies.

Dataset	Labelled	Unlabelled	Target
Kryptonite-20	12,000	10,000	90%
Kryptonite-30	20,000	10,000	90%
Kryptonite-40	32,000	10,000	90%
Kryptonite-40-Adapt	4,000	10,000	90%

4.2. Target Performance for Successful Models

Kryptonite-2.0 was criticised for lowering its targets as the dimension grew. For Kryptonite-3.0 we hold every dataset to the same standard: a model is considered to have solved a dataset if it reaches 90% accuracy, for every value of n and for the adaptation task. This threshold allows us to gauge performance consistently across all of our settings, and as we will see, it is far beyond the reach of every model we consider.

4.3. Performance of Basis Expanded Logistic Regression

In order to test logistic regression models with polynomial basis expansion, we first split our data into 60% training data, 20% validation data, and 20% testing data. We then train our model using the stochastic average gradient (SAG) approach which is equivalent to what is described in Equation (1). We use basic weight-decay regularization with a constant of 1/0.85 and use sci-kit-learn implementations to ensure that our figures and analysis are not the result of faulty implementation.

In Figure 1, we visualize the performance of logistic regression with polynomial basis expansion for $n = 20$, $n = 30$, and $n = 40$. Explicit features no longer fit in memory beyond degree 4, so for every degree from 1 to 10 we fit the model in its dual form using the kernel trick described in Section 3.2, with the kernel normalised so that $k(\mathbf{x}, \mathbf{x}) = 1$ and the same regularization constant. At degree 10 this corresponds to more than 30 million features for $n = 20$ and more than ten billion for $n = 40$. For every value of n and every degree, the polynomial regression model does not come even close to the target accuracy, plotted on the y-axis as a dotted line. The best result, 58.4% at $n = 20$ with degree 10, is barely better than random guessing, and going from degree 2 to degree 10 adds at most 3.1 percentage points.

4.4. Does More Data Help?

A common response to Kryptonite-2.0 was that our datasets were simply too small. To test this claim, we train quadratic ($d = 2$) logistic regression on 25%, 50% and 100% of each training split and evaluate on the same test set. As shown in Figure 2, quadrupling the amount of data improves accuracy by at most 3 percentage points (from 52.2% to 54.8% for $n = 20$), and by only 1.2 percentage points for $n = 40$. At this rate, reaching our target would require an astronomical amount of data, which we take as strong evidence that the difficulty of Kryptonite-3.0 is intrinsic and cannot be solved by scale.

4.5. Performance of KryptoNet-40

We hold out 20% of Kryptonite-40-Adapt (800 examples) for testing and use the remaining 3,200 examples to retrain the final layer. As shown in Figure 3, KryptoNet-40 used

Math for ML (70015) Coursework Prompt

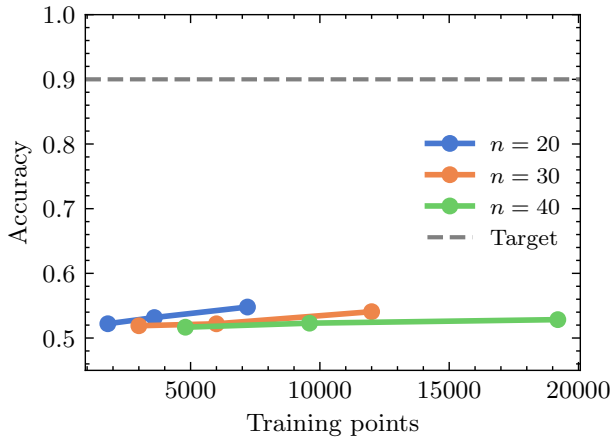


Figure 2. Accuracy of quadratic logistic regression as the number of training examples grows. Quadrupling the training data improves accuracy by at most 3 percentage points, suggesting that Kryptonite-3.0 cannot be solved by simply collecting more data.

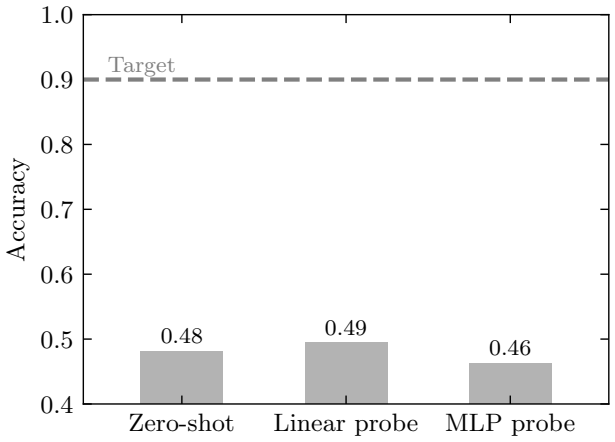


Figure 3. Accuracy of KryptoNet-40 on Kryptonite-40-Adapt. The pretrained model used zero-shot, and linear and MLP probes trained on its frozen representation, are all worse than random guessing, far below the target accuracy (dashed).

zero-shot achieves an accuracy of 0.481 despite reaching 95% on its pretraining task. Retraining the final layer on top of the frozen network does no better (0.495), and neither does replacing it with a one-hidden-layer MLP (0.464). This echoes our finding with GPT-2 embeddings in Kryptonite-2.0: the pretrained representation appears to destroy any even slightly useful information that existed in the original feature space, linear or otherwise. If a model pretrained on 200,000 examples of a closely related task cannot provide a useful representation, it is hard to see what pretraining could.

5. Future Works

Different Machine Learning Models In this work, we primarily examined the performance of logistic regression for our binary classification task. While logistic regression is well-suited for linearly separable data, it may be suboptimal for complex patterns. Future studies could explore alternative models, such as neural networks trained from scratch, which are known for their capacity to capture intricate, non-linear relationships through deep representations. Additionally, tree-based models or clustering-based approaches could provide a richer structure for capturing decision boundaries. Comparative analysis across these models could yield valuable insights into the trade-offs in accuracy, interpretability, and computational efficiency.

Different Adaptation Strategies Our foundation model experiments used KryptoNet-40 only as a fixed feature extractor, as we expected that updating its hidden layers on a few thousand examples would overfit. Parameter-efficient fine-tuning methods such as low-rank adaptation (LoRA) (Hu et al., 2022), which restrict the update to each weight matrix to a low-rank matrix, were designed to reduce this risk, although we doubt that a representation which performs worse than random guessing can be rescued. A careful study of such methods, together with an analysis of what exactly differs between Kryptonite-40-Pre and Kryptonite-40-Adapt, could be an interesting direction.

Scaling Analysis Our study of training set size considered a single model and three dataset sizes. A more systematic analysis of how the accuracy of different models scales with the number of training examples, for example by plotting learning curves across several orders of magnitude, could reveal whether any model class exhibits qualitatively different behaviour on Kryptonite-3.0.

Convergence Analysis The convergence properties of gradient descent in logistic regression are fundamental for efficient training. In this study, we selected hyperparameters based on empirical tuning, yet the theoretical implications of these choices were not fully explored. Future work could undertake a rigorous convergence analysis, investigating the sensitivity of convergence rates to learning rates, batch sizes, and initialization schemes.

Function Approximation Our findings challenge the universal function approximation theory, suggesting that the basis expansions chosen in our models may not fully capture the target function. This raises questions about the adequacy of the basis functions in our analysis. A detailed

Math for ML (70015) Coursework Prompt

investigation into the theoretical properties of different basis expansions, for example the degree a polynomial would need to represent our labels and the number of examples needed to fit it, could provide clarity on whether the function space we explored is sufficiently expressive.

`Predictive Uncertainty` Accounting for noise is critical in developing reliable machine learning models, as uncertainty quantification directly impacts the robustness of predictions. Incorporating probabilistic models that explicitly model noise could yield improved predictive performance, particularly in noisy environments. Evaluating these models with predictive uncertainty metrics, such as confidence intervals or entropy measures, would provide a benchmark for assessing model reliability.

References

Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L. D., Monfort, M., Muller, U., Zhang, J., et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Bubeck, S., Chandak, V., Eldan, R., Gehrke, J., Hoi, S., Hou, L., Krawczuk, P., Laskin, M., Li, Y. T., Terzi, E., et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Chollet, F. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

De Branges, L. The stone-weierstrass theorem. *Proceedings of the American Mathematical Society*, 10(5):822–824, 1959.

Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., Cui, C., and Corrado, G. A guide to deep learning in healthcare. *Nature medicine*, 25(1): 24–29, 2019.

Heaton, J., Polson, N., and Witte, J. Deep learning for finance: deep portfolios. *Applied Stochastic Models in Business and Industry*, 33(1):3–12, 2017.

Hornik, K. Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257, 1991.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Küttler, H., Germain, T., Teh, Y. W., Rocktäschel, T., and Grefenstette, E. The nethack learning environment. In *NeurIPS*, 2020.

Moskvichev, A., Odouard, V. V., and Mitchell, M. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain. *arXiv preprint arXiv:2305.07141*, 2023.

Quinn, H. and Luther, L. Kryptonite-2.0: A simple end to machine learning hype?, 2025. Mathematics for Machine Learning (70015) coursework prompt.

Sawada, T., Paleka, D., Havrilla, A., Tadepalli, P., Vidas, P., Kranias, A., Nay, J. J., Gupta, K., and Komatsuzaki, A. Arb: Advanced reasoning benchmark for large language models. *arXiv preprint arXiv:2307.13692*, 2023.

Math for ML (70015) Coursework Prompt